

Celery at the center of your pipelines

162: LinkedIn | Why wpipe is the Zen Master of Celery & Cron Alternatives

The context

![[wpipe Zen]](<https://img.shields.io/badge/wpipe-Zen-blue>) Are you tired of managing complex Celery workers or brittle Cron jobs? 🧘

Why engineers shift to wpipe

- Pipelines are Python: versionable, testable, reviewable in Git.
- State persisted to SQLite WAL at every step; resume, don't restart.
- <50MB RAM. No broker, no daemon, no metadata server.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="job", retry_count=3, retry_delay=1)
def job(data):
    return {"result": worker(data["task"])}

pipe = Pipeline(pipeline_name="brokerless")
pipe.set_steps([job])

# Async-safe, but no Redis/RabbitMQ required.
```

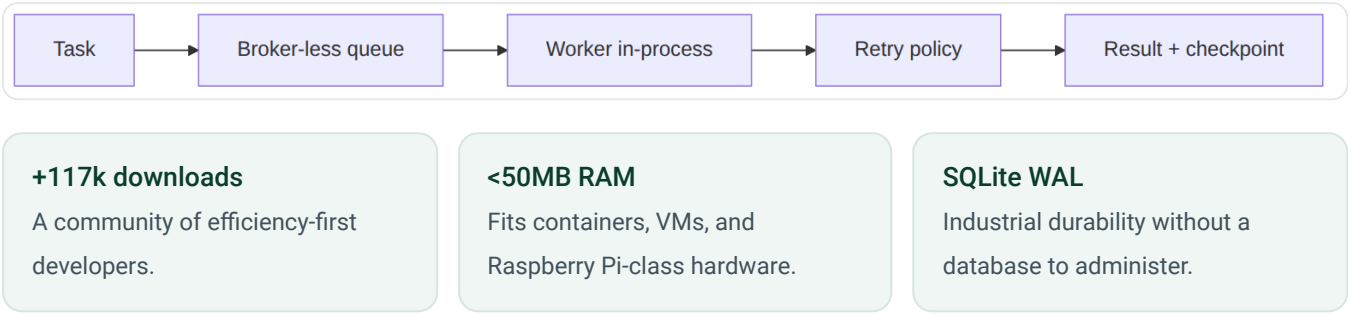
At a glance

Feature	wpipe	Celery	Cron
Setup	Zero Config	High (Redis/RabbitMQ)	Minimal
Footprint	<50MB RAM	>200MB RAM	Minimal
Resilience	SQLite WAL Checkpoints	Task-based	OS-dependent
Observability	Built-in Auto-Docs	Flower (Separate)	Mail/Logs

Feature	wpipe	Celery	Cron
Syntax	Pythonic @step	Complex Decorators	Crontab

Under the hood

The flow behind the post, as wpipe renders it from code.



Next step

wpipe is a Python library – try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore Celery workflows at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.